

AGENTIC AI CAPSTONE PROGRAM

Day 2 Lab Guide Build & Validate

Multi-Agent Orchestration, MCP & A2A Integration, a Voice Agent Lab, Retrieval Augmented Grounding, an Agent Test Harness, and Observability — built on top of the Day 1 Rule Engine

VM ONLY

Azure AI Foundry

VS Code

LangGraph

ChromaDB

Notebook file: 2.ipynb (depends on 1.ipynb in the same folder)

Scenario: Automated Customer Payment & Billing Reconciliation System (GreenGrid Utilities)

Format: Self-contained lab — environment setup, business context, and a full cell-by-cell code walkthrough

Environment: All work is performed on the assigned training Virtual Machine only

Contents

1. Environment Setup (VM, Azure AI Foundry, VS Code, additional packages)

2. The Business Scenario Behind This Notebook

3. Cell-by-Cell Walkthrough of 2.ipynb (21 cells)

HOW TO USE THIS LAB GUIDE

This notebook builds directly on Notebook 1 — make sure 1.ipynb has been completed and sits in the same folder before you start, since Cell 1 of this notebook runs it automatically. Work through Section 1 first for the extra packages this lab needs, Section 2 for the Day 2 business context (Build & Validate labs B1–B6), then Section 3 cell by cell.

Section 1 — Environment Setup (Perform Every Step on the VM)

DO THIS ON THE VIRTUAL MACHINE ONLY

Continue working inside the same training Virtual Machine and the same project folder you used for Day 1. This notebook literally runs 1.ipynb as its first step, so both files must live side by side, and the packages you install below must go into the same `.venv` you already created.

1.1 Confirm Day 1 Is Complete

Step 1 — Check your folder

1. Open the same project folder you used for Day 1 in VS Code on your VM.
2. Confirm both `1.ipynb` and `2.ipynb` are present in that folder — Cell 1 of this notebook executes `%run 1.ipynb`, which only works if the file is sitting right next to it.
3. You do not need to re-run 1.ipynb yourself first — the `%run` magic inside 2.ipynb's first cell does that for you automatically, every time you run this notebook from a fresh kernel.

1.2 Reuse Your Existing Azure AI Foundry Project

Day 2 reuses the exact same Foundry project, hub, and `gpt-5` deployment you created on Day 1 — there is nothing new to provision in the Foundry portal. If you are starting a fresh VM session, confirm your project still shows the deployment under **Models + endpoints** before continuing.

1.3 Set Your Credentials (Same Three Variables as Day 1)

Step 2 — Confirm the first cell of 1.ipynb still has your real values

Because this notebook runs 1.ipynb internally, the credential cell you added at the very top of 1.ipynb on Day 1 (`AZURE_AI_FOUNDRY_ENDPOINT`, `AZURE_AI_FOUNDRY_API_KEY`, `MODEL_DEPLOYMENT_NAME`) is inherited automatically — there is no separate credential cell to add here.

THE GROUNDED REASONING CELL DEGRADES GRACEFULLY WITHOUT CREDENTIALS

One cell in this notebook (the Azure AI Foundry grounded-reasoning node) checks whether `AZURE_AI_FOUNDRY_ENDPOINT` and `AZURE_AI_FOUNDRY_API_KEY` are actually set as environment variables. If they aren't, it prints a clear message and falls back to the Day 1 keyword-heuristic result instead of raising an error. You can complete this entire lab and see meaningful output even before your Foundry credentials are wired up as environment variables — wiring them up simply upgrades that one cell from a rule-based guess to a real grounded LLM call.

1.4 Install the Additional Day 2 Packages

Step 3 — Add ChromaDB, FastMCP, and Microsoft Agent Framework to your .venv

Open the integrated terminal in VS Code (Ctrl+`), make sure your Day 1 `.venv` is activated, and install the packages this lab adds on top of Day 1:

Windows VM (PowerShell)

```
.venv\Scripts\Activate.ps1  
pip install chromadb fastmcp agent-framework
```

Linux VM (bash)

```
source .venv/bin/activate  
pip install chromadb fastmcp agent-framework
```

These four packages map directly onto the Day 2 labs: **chromadb** is the vector store for the Retrieval Augmented Agents module, **fastmcp** is the standalone Model Context Protocol server framework for the MCP & A2A Integration Lab, and **agent-framework** is Microsoft's production Agent Framework for the LangGraph & Microsoft Agent Framework Lab.

IF A PACKAGE FAILS TO INITIALIZE WHEN YOU RUN THE NOTEBOOK

Both the Microsoft Agent Framework cell and the MCP tool-server cell are wrapped in try/except blocks. If a package is missing or a required environment variable isn't set, you will see a friendly one-line message explaining what's missing rather than a crash — the rest of the notebook keeps running normally. This is intentional: it lets you work through the whole lab even before every optional integration is fully configured, and it is exactly the same defensive pattern you saw throughout Day 1.

1.5 Open 2.ipynb in VS Code and Select the Kernel

Step 4 — Open and run

1. Click **2.ipynb** in the Explorer panel to open it in the Notebook editor.
2. In the top-right of the notebook, confirm **Select Kernel** is still pointed at the same **.venv** interpreter you used for Day 1.
3. Run cells top to bottom with Shift+Enter, following the walkthrough in Section 3 of this guide. Expect the very first cell to take a few seconds longer than usual — it is re-executing the entire Day 1 notebook behind the scenes.

Section 2 — The Business Scenario Behind This Notebook

Day 2 keeps the exact same GreenGrid Utilities payment-reconciliation scenario from Day 1, but shifts the focus from Track A (Architecture & Design) to Track B: **Build & Validate**. Each of the six hands-on labs in this notebook takes one architectural idea from Day 1 and turns it into working, testable code.

2.1 The Six Day 2 Labs

Lab	What it adds
B1 — LangGraph & Microsoft Agent Framework Lab	Wraps the Day 1 node functions as named agents under an explicit Supervisor, and initializes a real Microsoft Agent Framework Agent object alongside it.
B2 — MCP & A2A Integration Lab	Stands up a FastMCP tool server exposing a governed <code>query_billing_record</code> tool, and models the A2A message contract agents use to hand off state.
B3 — Voice Agent Lab	Simulates a speech-to-text transcript from an inbound customer call and feeds it into the exact same Remittance Processing node from Day 1.
B4 — Testing & Evaluating Agents	A small assertion-based regression suite that checks the Day 1 examples still resolve to the states you expect.
B5 — Deploying & Operating Agents	A deployment manifest describing how this graph would run in Azure Container Apps, plus a batch "operate" loop.
B6 — Observability Deep Dive	Aggregates the telemetry already being written to <code>system.log</code> into simple operational metrics.

2.2 A4: Retrieval Augmented Agents

Alongside the six B-labs, this notebook also builds out module A4 from the Day 1 architecture: a persistent **ChromaDB** vector store holding unstructured dispute notes and outage logs, a semantic search node, and a Foundry-grounded reasoning node that maps retrieved evidence to a standardized adjustment code. This is the direct upgrade path for the keyword-only Remittance Processing heuristic you built in Notebook 1.

A NOTE ON THE EMBEDDING FUNCTION USED IN THIS LAB

Rather than downloading a large embedding model from the internet, this lab uses a small, self-contained hashing-based embedding function so the vector search works reliably offline and starts instantly. In a production deployment you would swap this for a real embedding model (for example, Azure OpenAI's `text-embedding-3-large`) for materially better retrieval quality — the node interface stays exactly the same either way.

2.3 How This Fits the Agent Architecture from Day 1

Recall the five specialist agents from the Day 1 architecture spec: the Ingestion Agent, Matching Agent, RAG Reasoning Agent, Guardrail & Reporting Agent, and the Supervisor that coordinates them all. This notebook is where you actually instantiate that pattern in code — the `ReconciliationAgent` wrapper class and `supervisor_orchestrate` function you will build in Cells 2–3 are a direct, working implementation of that design.

WHY THIS CONTEXT MATTERS WHILE YOU CODE

Whenever a cell mentions "GreenGridBillingTools", "IngestionAgent", or a document ID like DOC-MER-001, it is exercising one of the Day 2 labs described above — recognizing which lab you are in makes it much easier to predict what a cell's output should look like before you run it.

Section 3 — Cell-by-Cell Walkthrough of 2.ipynb

This notebook has 21 code cells. Run them in order, top to bottom, in VS Code on your VM. Each card below shows the cell number, a plain-language explanation of what it does and why, the exact code, and the output you should expect to see when you run it yourself.

Cell 1 — Inherit Notebook 1 with %run

This is the dependency-integration cell every Day 2 and Day 3 notebook starts with. `%run 1.ipynb` is an IPython magic command that executes every cell of Notebook 1 in this notebook's own namespace — so after this cell finishes, every schema, node function, sample payload, and the compiled `app` graph from Day 1 is available here exactly as if you had typed it yourself. You'll see the entire Day 1 output stream past as this cell runs — that's expected.

CODE

```
# Inherit state schemas, node functions, JSONStateDeltaLogger, and the compiled
# `app` graph from Day 1. Run this cell with 1.ipynb saved in the same folder.
%run 1.ipynb
```

OUTPUT

```
{"timestamp": "2026-08-16T11:35:36.795865+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",
"transaction_id": "TXN-MER-6120", "account_number": "ACC-77210", "state_transition": {"from":
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 4, "adjustment_code": null},
"delta_payload": {"customer_name": "Meridian Manufacturing", "bill_id": "BILL-4471",
"amount_paid": 482.5, "match_type": "2-Way", "status_reason": "Perfect 2-Way Match
Successful"}}
```

Telemetry event logged successfully.

--- System Log File Contents ---

```
{"timestamp": "2026-08-16T11:35:36.795865+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",
"transaction_id": "TXN-MER-6120", "account_number": "ACC-77210", "state_transition": {"from":
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 4, "adjustment_code": null},
"delta_payload": {"customer_name": "Meridian Manufacturing", "bill_id": "BILL-4471",
"amount_paid": 482.5, "match_type": "2-Way", "status_reason": "Perfect 2-Way Match
Successful"}}
```

Data Ingestion Result:

- Data Ingestion validated payment TXN-6055 from channel 'ACH_BANK_TRANSFER'.

Remittance Processing Result:

- Remittance Processing extracted claimed_amount=50.0 keyword_hint_code=U01 from dispute note DSP-330.

Adjustment Code Hint: U01

Example A (Perfect Match) Result:

Priority Rule Matched: 1

Match Type: 2-Way

- State initialized to OPEN.
- Priority Rule 1 Matched: Account + Bill Number + Amount.

Smart Matching Result:

- State initialized to OPEN.
- No Billing AR record available; no rule can be evaluated.
- Smart Matching suggests bill BILL-4471 as the probable match (confidence=0.90). Routed for analyst confirmation.

Ranked Candidates:

```
{'bill_id': 'BILL-4471', 'confidence': 0.9034, 'amount_score': 0.9918, 'name_score': 0.85,
'date_score': 0.7333}
{'bill_id': 'BILL-9081', 'confidence': 0.1453, 'amount_score': 0.0, 'name_score': 0.1579,
'date_score': 0.6}
```

Dynamic Router Decision for Example B: 'deduce_adjustment'

--- Example B Final Output ---

Final State: PARTIAL MATCH

Priority Rule Matched: 6

Adjustment Code: U01 (Meter Reading Dispute)

Approved Settlement: \$560.00

Held Variance: \$50.00

Audit Notes:

- State initialized to OPEN.
- Data Ingestion validated payment TXN-6055 from channel 'ACH_BANK_TRANSFER'.
- Remittance Processing extracted claimed_amount=50.0 keyword_hint_code=U01 from dispute note DSP-330.
- Priority Rule 6 Matched: Account + Bill Number + Amount, reinforced by dispute note (3-Way).
- Short payment of \$50.00 identified. Mapped to code U01 (Meter Reading Dispute).
- Variance of \$50.00 exceeds \$5.00 tolerance. End State: PARTIAL MATCH.

Dynamic Router Decision for Example D: 'handle_unmatched'

--- Example D Final Output ---

Final State: UAC

Audit Notes:

- State initialized to OPEN.
- Data Ingestion validated payment TXN-5000 from channel 'ACH_BANK_TRANSFER'.
- No Billing AR record available; no rule can be evaluated.
- Customer name recognized but no bill/account reference found. End State: UAC.

```
{"timestamp": "2026-08-16T11:35:36.827353+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",  
"transaction_id": "TXN-6055", "account_number": "ACC-77210", "state_transition": {"from":  
"OPEN", "to": "PARTIAL MATCH"}, "rule_context": {"priority_rule": 6, "adjustment_code": "U01"},  
"delta_payload": {"approved_settlement": 560.0, "held_variance": 50.0, "routing_path":  
"deduce_adjustment"}}
```

```
{"timestamp": "2026-08-16T11:35:36.828795+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",  
"transaction_id": "TXN-5000", "account_number": null, "state_transition": {"from": "OPEN",  
"to": "UAC"}, "rule_context": {"priority_rule": null, "adjustment_code": null},  
"delta_payload": {"routing_path": "handle_unmatched", "reason": "No bill/account reference on  
payment"}}
```

Node execution telemetry successfully appended to system.log.

StateGraph builder initialized with UtilityReconciliationState schema.

All 6 processing nodes successfully added to StateGraph builder.

Graph execution edges and dynamic conditional routing configured successfully.

StateGraph compiled successfully into an executable runnable application.

--- Example A Graph Execution Output ---

Final State: CLOSED

Priority Rule Matched: 1

Match Type: 2-Way

Approved Settlement: \$482.50

Audit Log Trail:

- State initialized to OPEN.
- Data Ingestion validated payment TXN-11000 from channel 'ACH_BANK_TRANSFER'.
- Remittance Processing: no dispute/adjustment note attached.
- Priority Rule 1 Matched: Account + Bill Number + Amount.
- No short payment detected; full billed amount settled.
- Full match with zero variance. End State: CLOSED.

--- Example C Graph Execution Output (Tolerance) ---

Final State: CLOSED

Priority Rule Matched: 3

Auto Write-Off Amount: \$2.00

Approved Settlement: \$150.00

Audit Log Trail:

- State initialized to OPEN.
- Data Ingestion validated payment TXN-9002 from channel 'ONLINE_BILLPAY'.
- Remittance Processing: no dispute/adjustment note attached.
- Priority Rule 3 Matched: Account + Bill Number + Bill Date + Amount.
- Short payment of \$2.00 identified. Mapped to code None (None).
- Variance of \$2.00 is <= \$5.00 tolerance. Auto-written off. End State: CLOSED.

```
{"timestamp": "2026-08-16T11:35:37.389507+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",  
"transaction_id": "TXN-11000", "account_number": "ACC-77210", "state_transition": {"from":  
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 1, "adjustment_code": null},  
"delta_payload": {"approved_settlement": 482.5, "execution_type":  
"Compiled_StateGraph_Invoke"}}
```

```
{"timestamp": "2026-08-16T11:35:37.391105+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",  
"transaction_id": "TXN-9002", "account_number": "ACC-88510", "state_transition": {"from":  
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 3, "adjustment_code": null},
```

```
"delta_payload": {"auto_write_off": 2.0, "execution_type": "Compiled_StateGraph_Invoke"}}
```

Compiled graph execution results logged to system.log successfully.

```
=====
```

DAY 1 CAPABILITIES BUILT

```
=====
```

- A1 Agent Architecture: deterministic Planner-Executor nodes
- A2 Multi-Agent Orchestration: compiled LangGraph StateGraph
- Data Ingestion, Remittance Processing, Payment Matching, and Smart Matching use cases all implemented and tested
- 6 End States supported: OPEN, PARTIAL MATCH, CLOSED, UAC, UIC, QUERY

Next: Notebook 2 adds MCP/A2A integration, vector RAG grounding, a voice-intake lab, and an agent evaluation harness (Day 2 / Track B).

Cell 2 — B1: Wrap the Day 1 nodes as named agents

This defines a minimal `ReconciliationAgent` wrapper class — just a name, a role label, and a callable — and instantiates four of them (`IngestionAgent`, `MatchingAgent`, `AdjustmentAgent`, `ExceptionAgent`), each one simply delegating to the plain node function you already built in Notebook 1. This is the code-level realization of the Agent Architecture Patterns from Day 1's A1 module: each agent has one narrow, auditable job.

CODE

```
# =====
# B1: LangGraph & Microsoft Agent Framework Lab
# Wraps the Day 1 nodes as explicitly named agents and adds a Supervisor
# routine that decides which specialist agent runs next -- the Supervisor
# pattern documented in the capstone architecture spec (Section 2).
# =====
class ReconciliationAgent:
    # A minimal agent wrapper: a name, a role description, and a callable.
    def __init__(self, name: str, role: str, fn):
        self.name = name
        self.role = role
        self.fn = fn

    def run(self, state: Dict[str, Any]) -> Dict[str, Any]:
        print(f"[{self.name}] executing ({self.role}) ...")
        return self.fn(state)

ingestion_agent = ReconciliationAgent("IngestionAgent", "Planner-Executor",
data_ingestion_node)
matching_agent = ReconciliationAgent("MatchingAgent", "Deterministic Executor",
execute_priority_rules_node)
adjustment_agent = ReconciliationAgent("AdjustmentAgent", "Deterministic Executor",
adjustment_discrepancy_node)
exception_agent = ReconciliationAgent("ExceptionAgent", "Supervisor sub-routine",
unmatched_exception_node)

print("Registered agents:", [a.name for a in [ingestion_agent, matching_agent,
adjustment_agent, exception_agent]])
```

OUTPUT

```
Registered agents: ['IngestionAgent', 'MatchingAgent', 'AdjustmentAgent', 'ExceptionAgent']
```

Cell 3 — Define and test the Supervisor orchestration function

supervisor_orchestrate is the Supervisor pattern in action: it owns the routing decision, calling **IngestionAgent**, then Remittance Processing, then **MatchingAgent**, then checking **route_reconciliation_workflow** to decide whether **AdjustmentAgent** or **ExceptionAgent** runs next. Running it against the Example B payload from Day 1 should print each agent firing in order and land on PARTIAL MATCH.

CODE

```
def supervisor_orchestrate(state: Dict[str, Any]) -> Dict[str, Any]:
    # Supervisor pattern: owns the routing decision across specialist agents.
    state = ingestion_agent.run(state)
    state = remittance_processing_node(state)
    state = matching_agent.run(state)

    route = route_reconciliation_workflow(state)
    print(f"[Supervisor] routing decision: '{route}'")

    if route == "deduce_adjustment":
        state = adjustment_agent.run(state)
        state = tolerance_auto_write_off_node(state)
    else:
        state = exception_agent.run(state)

    return state

try:
    demo_state = supervisor_orchestrate(
        create_initial_reconciliation_state(sample_payment_b, sample_billing_b,
        sample_dispute_b)
    )
    print(f"\nSupervisor-orchestrated Final State: {demo_state['current_state']}")
except Exception as err:
    print(f"Supervisor orchestration failed: {err}")
```

OUTPUT

```
[IngestionAgent] executing (Planner-Executor) ...
[MatchingAgent] executing (Deterministic Executor) ...
[Supervisor] routing decision: 'deduce_adjustment'
[AdjustmentAgent] executing (Deterministic Executor) ...

Supervisor-orchestrated Final State: PARTIAL MATCH
```

Cell 4 — Initialize a Microsoft Agent Framework Agent

Microsoft Agent Framework (MAF) is Microsoft's production successor to AutoGen, giving agents a consistent Python/.NET API with native MCP and A2A interoperability. This cell wraps a small `matching_tool` function as a MAF `Agent` using `OpenAIChatClient`. Because `OpenAIChatClient()` needs a real endpoint/key to fully initialize, expect this cell to print a friendly "not initialized in this environment" message if you haven't wired your Azure OpenAI credentials into the environment yet — that is the try/except working as designed, not a failure of your setup.

CODE

```
# Microsoft Agent Framework (MAF) is Microsoft's production successor to AutoGen,
# giving agents a consistent Python/.NET API and native A2A/MCP interoperability.
# pip install agent-framework
# We wrap MatchingAgent as a Microsoft Agent Framework Agent so it can be
# composed alongside other MAF-native agents (Azure OpenAI, Foundry, etc.).
try:
    from agent_framework import Agent
    from agent_framework.openai import OpenAIChatClient

    def matching_tool(account_number: str, bill_amount: float, paid_amount: float) -> str:
        # Reports whether a payment matches a bill within $0.01.
        return "MATCH" if abs(bill_amount - paid_amount) < 0.01 else "NO_MATCH"

    maf_matching_agent = Agent(
        name="MatchingAgent",
        client=OpenAIChatClient(),
        tools=[matching_tool],
        instructions="You verify utility bill payments using the matching_tool.",
    )
    print("Microsoft Agent Framework MatchingAgent initialized:", maf_matching_agent.name)
except Exception as err:
    print(f"Microsoft Agent Framework agent not initialized in this environment "
          f"(expected without a configured OpenAI/Azure key): {err}")
```

OUTPUT

```
Microsoft Agent Framework agent not initialized in this environment (expected without a
configured OpenAI/Azure key): Exactly one of 'base_url', 'endpoint' must be provided, but none
was set.
```

Cell 5 — B2: Define the FastMCP billing tool server

This is the MCP half of the Integration Lab. `FastMCP("GreenGridBillingTools")` creates an MCP tool server, and the `@billing_mcp.tool` decorator registers `query_billing_record` as a governed, parameterized tool over an in-memory mock of the D365 Billing AR table — the whole point of MCP being that an agent can only call this one narrow function, never issue arbitrary SQL against the real database. In production this server runs as its own standalone process; here we simply define it so you can see the tool-calling contract.

CODE

```
# =====
# B2: MCP & A2A Integration Lab
# MCP gives agents structured, permissioned tool access to the Billing AR
# system and payment ledger -- never raw SQL, never direct DB exposure.
# pip install fastmcp
# We define the tool server here; in production it runs as its own process
# (`python billing_mcp_server.py`) and agents connect to it as an MCP client.
# =====
try:
    from fastmcp import FastMCP

    billing_mcp = FastMCP("GreenGridBillingTools")

    # In-memory mock of the D365 Billing AR table for the lab
    _MOCK_BILLING_AR = {
        "BILL-4471": {"account_number": "ACC-77210", "customer_name": "Meridian Manufacturing",
"total_amount": 610.00},
        "BILL-2290": {"account_number": "ACC-88510", "customer_name": "Coastal Retail Group",
"total_amount": 150.00},
    }

    @billing_mcp.tool
    def query_billing_record(bill_id: str) -> dict:
        # Look up a single open bill by bill_id. Read-only, parameterized -- no raw SQL.
        return _MOCK_BILLING_AR.get(bill_id, {"error": "not_found"})

    print("FastMCP server 'GreenGridBillingTools' defined with 1 registered tool:
query_billing_record.")
    print("(In production this runs standalone via `python billing_mcp_server.py`; not started
inline here.)")
except Exception as err:
    print(f"MCP tool server setup skipped: {err}")
```

OUTPUT

```
FastMCP server 'GreenGridBillingTools' defined with 1 registered tool: query_billing_record.
(In production this runs standalone via `python billing_mcp_server.py`; not started inline
here.)
```

Cell 6 — Call the MCP tool directly

Confirms the tool actually works by calling `query_billing_record("BILL-4471")` and printing the result. Because the `@tool` decorator leaves the underlying function directly callable, you can test it locally exactly like this before ever standing up the full MCP transport layer.

CODE

```
try:
    # .fn reaches the underlying function directly for local testing,
    # without spinning up the stdio/SSE transport layer.
    _fn = query_billing_record.fn if hasattr(query_billing_record, "fn") else
    query_billing_record
    result = _fn("BILL-4471")
    print("MCP tool call result for BILL-4471:", result)
except Exception as err:
    print(f"MCP tool invocation failed: {err}")
```

OUTPUT

```
MCP tool call result for BILL-4471: {'account_number': 'ACC-77210', 'customer_name': 'Meridian
Manufacturing', 'total_amount': 610.0}
```

Cell 7 — Route Payment Matching through the MCP tool

This demonstrates what "MCP as an integration contract" means in practice for the Payment Matching use case: instead of reading a locally-held Python dict, `mcp_backed_priority_match` fetches the billing record through the governed `query_billing_record` tool and then runs it through the exact same `execute_priority_rules_node` from Day 1. The Matching Agent never touches the billing system directly — it only ever sees what the tool returns.

CODE

```
# Demonstrate the Payment Matching use case going through the MCP tool
# instead of a locally-held dict -- this is what "MCP as an integration
# contract" means in practice: the Matching Agent never touches D365 directly.
def mcp_backed_priority_match(payment: Dict[str, Any]) -> Dict[str, Any]:
    bill_id = payment.get("reference_number")
    _fn = query_billing_record.fn if hasattr(query_billing_record, "fn") else
    query_billing_record
    billing_record_raw = _fn(bill_id) if bill_id else {"error": "not_found"}

    if "error" in billing_record_raw:
        return {"matched": False, "reason": "Bill not found via MCP tool."}

    billing_ar = {
        "bill_id": bill_id,
        "account_number": billing_record_raw["account_number"],
        "customer_name": billing_record_raw["customer_name"],
        "meter_id": None,
        "bill_date": payment.get("payment_date", ""),
        "total_amount": billing_record_raw["total_amount"],
        "is_open": True,
    }
    state = create_initial_reconciliation_state(payment, billing_ar, None)
    state = execute_priority_rules_node(state)
    return {"matched": state["priority_rule_matched"] is not None, "state": state}

try:
    mcp_match_result = mcp_backed_priority_match(sample_payment_b)
    print("MCP-backed Payment Matching result:")
    print(f"  Matched: {mcp_match_result['matched']}")
    if mcp_match_result["matched"]:
        print(f"  Priority Rule: {mcp_match_result['state']['priority_rule_matched']}")
except Exception as err:
    print(f"MCP-backed matching demo failed: {err}")
```

OUTPUT

```
MCP-backed Payment Matching result:
Matched: True
Priority Rule: 3
```

Cell 8 — Define the A2A message contract and simulate a hand-off

A2A (Agent2Agent) is the open protocol for structured hand-offs between agents, potentially built on different frameworks. A full A2A server needs an Agent Card, an Agent Executor, and an ASGI app — for this lab, `A2AMessage` models just the message contract itself: a task ID, the sending and receiving agent names, and a state snapshot. `a2a_handoff` then simulates the Ingestion Agent handing state to the Matching Agent using that contract.

CODE

```
# A2A (Agent2Agent) is the open protocol for agent-to-agent hand-offs across
# frameworks and vendors. pip install a2a-sdk
# A full A2A server needs an AgentCard + AgentExecutor + ASGI app (Starlette/uvicorn).
# For this lab we model the *message contract* -- the structured payload one
# agent sends another -- which is the part every agent in our graph actually uses.
from dataclasses import dataclass
import uuid

@dataclass
class A2AMessage:
    task_id: str
    from_agent: str
    to_agent: str
    state_snapshot: Dict[str, Any]
    message_type: str = "STATE_HANDOFF"

    @classmethod
    def create(cls, from_agent: str, to_agent: str, state_snapshot: Dict[str, Any]) ->
"A2AMessage":
        return cls(task_id=str(uuid.uuid4()), from_agent=from_agent, to_agent=to_agent,
state_snapshot=state_snapshot)

def a2a_handoff(message: A2AMessage) -> Dict[str, Any]:
    print(f"[A2A] task={message.task_id[:8]} {message.from_agent} -> {message.to_agent}
({message.message_type})")
    return message.state_snapshot

try:
    handoff_msg = A2AMessage.create(
        "IngestionAgent", "MatchingAgent", {"transaction_id": "TXN-6055", "current_state":
"OPEN"}
    )
    received_state = a2a_handoff(handoff_msg)
    print("Received state at MatchingAgent:", received_state)
except Exception as err:
    print(f"A2A handoff simulation failed: {err}")
```

OUTPUT

```
[A2A] task=8db29b88 IngestionAgent -> MatchingAgent (STATE_HANDOFF)
Received state at MatchingAgent: {'transaction_id': 'TXN-6055', 'current_state': 'OPEN'}
```


Customers can also phone in a payment dispute. `simulate_ivr_transcript` stands in for a real speech-to-text service (like Azure AI Speech), returning a canned transcript for call reference CALL-9910. `voice_intake_node` then wraps that transcript into a normal `dispute_note` structure and feeds it straight into the same `remittance_processing_node` from Day 1 — no separate pipeline is needed for voice versus email/chat intake.

CODE

```
# =====
# B3: Voice Agent Lab
# Customers can also phone in a payment dispute. A voice agent's speech-to-text
# transcript becomes just another unstructured dispute note, feeding the exact
# same Remittance Processing node built in Notebook 1 -- no separate pipeline needed.
# =====
def simulate_ivr_transcript(raw_call_audio_ref: str) -> str:
    # Stand-in for a speech-to-text service (e.g., Azure AI Speech). In production
    # this calls the real STT endpoint; here we return a canned transcript for the lab.
    _FAKE_TRANSCRIPTS = {
        "CALL-9910": "Hi, I'm calling about my bill, I think the meter reading was way too high
"
        "this month, it looks estimated, can I get that corrected before I pay the
rest?",
    }
    return _FAKE_TRANSCRIPTS.get(raw_call_audio_ref, "")

def voice_intake_node(state: Dict[str, Any], call_ref: str) -> Dict[str, Any]:
    transcript = simulate_ivr_transcript(call_ref)
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))

    if transcript:
        updated_state["dispute_note"] = {
            "dispute_id": f"VOICE-{call_ref}",
            "account_number": state.get("payment_transaction", {}).get("account_number"),
            "unstructured_notes": transcript,
            "source_channel": "VOICE-IVR",
        }
        audit_notes.append(f"Voice Agent transcribed call {call_ref} into a dispute note.")
    else:
        audit_notes.append(f"Voice Agent found no transcript for call {call_ref}.")

    updated_state["audit_notes"] = audit_notes
    return updated_state

try:
    state_voice = create_initial_reconciliation_state(sample_payment_b, sample_billing_b, None)
    state_voice = voice_intake_node(state_voice, "CALL-9910")
    state_voice = remittance_processing_node(state_voice)
    print("Voice Agent Lab Result:")
    for note in state_voice["audit_notes"]:
        print(f" - {note}")
    print(f"Adjustment Code Hint from Voice Call: {state_voice['adjustment_code']}")
except Exception as err:
    print(f"Voice agent lab failed: {err}")
```

OUTPUT

```
Voice Agent Lab Result:
- State initialized to OPEN.
- Voice Agent transcribed call CALL-9910 into a dispute note.
- Remittance Processing extracted claimed_amount=None keyword_hint_code=U01 from dispute note
```

Cell 10 — A4: Initialize the persistent ChromaDB vector store

This sets up the Retrieval Augmented Agents module. Rather than relying on ChromaDB's default embedding model (which needs to download a file from the internet the first time it runs), this cell defines a small self-contained **SimpleHashingEmbeddingFunction** — a bag-of-words hash embedding with no external dependency — and uses it to create a persistent collection named **utility_dispute_docs** on disk at **./chroma_db**.

CODE

```
# =====
# A4: Retrieval Augmented Agents -- Persistent Vector Store
# Ingest dispute notes, outage logs, and field-technician notes into ChromaDB
# so the RAG Reasoning Agent can ground its adjustment-code decisions in
# retrieved evidence instead of the earlier keyword-only heuristic.
#
# We use a small self-contained hashing embedding function instead of
# ChromaDB's default downloaded ONNX model, so this lab runs fully offline
# and does not depend on reaching an external model registry. Swap in a
# real embedding model (Azure OpenAI text-embedding-3-large, etc.) in
# production for materially better retrieval quality.
# =====
try:
    import chromadb
    from chromadb import EmbeddingFunction
    import hashlib

    EMBEDDING_DIM = 256

    def _simple_hash_embedding(text: str, dim: int = EMBEDDING_DIM) -> List[float]:
        vec = [0.0] * dim
        for tok in re.findall(r"[a-z0-9]+", text.lower()):
            idx = int(hashlib.md5(tok.encode()).hexdigest(), 16) % dim
            vec[idx] += 1.0
        norm = sum(v * v for v in vec) ** 0.5
        return [v / norm for v in vec] if norm > 0 else vec

    class SimpleHashingEmbeddingFunction(EmbeddingFunction):
        def __init__(self) -> None:
            pass

        def __call__(self, input: List[str]) -> List[List[float]]:
            return [_simple_hash_embedding(t) for t in input]

        def name(self) -> str:
            return "simple_hashing_embedding"

    chroma_client = chromadb.PersistentClient(path="./chroma_db")
    dispute_collection = chroma_client.get_or_create_collection(
        name="utility_dispute_docs", embedding_function=SimpleHashingEmbeddingFunction()
    )
    print(f"ChromaDB collection ready: '{dispute_collection.name}' (existing docs: {dispute_collection.count()})")
except Exception as err:
    print(f"ChromaDB initialization failed: {err}")
```

OUTPUT

```
ChromaDB collection ready: 'utility_dispute_docs' (existing docs: 0)
```

Adds two unstructured documents to the collection: a field-technician note about a meter read for account ACC-77210, and an outage log for service point MTR-4402. Each document carries metadata (`adjustment_hint`) recording which adjustment code it's expected to support, and the ingestion event itself is logged to `system.log` just like every other state change in this capstone.

CODE

```
try:
    sample_docs = [
        {
            "id": "DOC-MER-001",
            "text": "Field Meter Read Note for ACC-77210: Technician confirms the prior cycle's
meter "
                "read was system-estimated; the actual meter read supports a $50 credit.",
            "metadata": {"account_number": "ACC-77210", "adjustment_hint": "U01"},
        },
        {
            "id": "DOC-MER-002",
            "text": "Outage Log for Service Point MTR-4402: 6-hour unplanned outage on 2026-07-
10; "
                "SLA credit of $18.40 applies.",
            "metadata": {"account_number": "ACC-77210", "adjustment_hint": "U02"},
        },
    ]

    dispute_collection.upsert(
        ids=[d["id"] for d in sample_docs],
        documents=[d["text"] for d in sample_docs],
        metadatas=[d["metadata"] for d in sample_docs],
    )

    telemetry = JSONStateDeltaLogger("system.log")
    telemetry.log_state_delta(
        transaction_id="VECTOR_INGEST",
        account_number=None,
        previous_state="EMPTY",
        new_state="INDEXED",
        priority_rule_applied=None,
        adjustment_code=None,
        delta_payload={"documents_indexed": len(sample_docs), "collection":
"utility_dispute_docs"},
    )
    print(f"Ingested {len(sample_docs)} documents into 'utility_dispute_docs'. "
          f"Collection now holds {dispute_collection.count()} documents.")
except Exception as err:
    print(f"Document ingestion failed: {err}")
```

OUTPUT

```
{"timestamp": "2026-08-16T11:35:39.051159+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",
"transaction_id": "VECTOR_INGEST", "account_number": null, "state_transition": {"from":
"EMPTY", "to": "INDEXED"}, "rule_context": {"priority_rule": null, "adjustment_code": null},
"delta_payload": {"documents_indexed": 2, "collection": "utility_dispute_docs"}}
Ingested 2 documents into 'utility_dispute_docs'. Collection now holds 2 documents.
```

`RAGUtilityState` extends the Day 1 `UtilityReconciliationState` with two new fields: `search_query` and `retrieved_documents`. `vector_search_node` queries the ChromaDB collection for the top 2 most similar documents to the search query and records their IDs, text, metadata, and distance scores onto the state.

CODE

```
class RAGUtilityState(UtilityReconciliationState):
    search_query: Optional[str]
    retrieved_documents: List[Dict[str, Any]]

def vector_search_node(state: Dict[str, Any]) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))
    query = state.get("search_query", "")

    try:
        results = dispute_collection.query(query_texts=[query], n_results=2)
        retrieved = []
        ids = results.get("ids", [[]])[0]
        docs = results.get("documents", [[]])[0]
        metas = results.get("metadatas", [[]])[0]
        dists = results.get("distances", [[]])[0]
        for doc_id, doc_text, meta, dist in zip(ids, docs, metas, dists):
            retrieved.append({"id": doc_id, "text": doc_text, "metadata": meta, "distance":
dist})

        updated_state["retrieved_documents"] = retrieved
        audit_notes.append(f"Vector Search retrieved {len(retrieved)} documents for query
'{query}'.")
    except Exception as err:
        updated_state["retrieved_documents"] = []
        audit_notes.append(f"Vector Search failed: {err}")

    updated_state["audit_notes"] = audit_notes
    return updated_state
```

This cell produces no printed output.

Cell 13 — Test the vector search against a meter-reading query

Runs the query "meter read estimated high dispute credit" against the two-document collection. You should see **DOC-MER-001** (the meter-reading note) come back as the closer match — a noticeably smaller distance than the outage-log document — confirming the retrieval is finding the semantically relevant note.

CODE

```
try:
    rag_state = dict(state_b)
    rag_state["search_query"] = "meter read estimated high dispute credit"
    rag_state = vector_search_node(rag_state)

    print("Vector Search Result:")
    for doc in rag_state["retrieved_documents"]:
        print(f" - [{doc['id']}] distance={doc['distance']:.4f} :: {doc['text'][:80]}...")
except Exception as err:
    print(f"Vector search test failed: {err}")
```

OUTPUT

```
Vector Search Result:
- [DOC-MER-001] distance=0.9672 :: Field Meter Read Note for ACC-77210: Technician confirms
the prior cycle's meter...
- [DOC-MER-002] distance=1.6595 :: Outage Log for Service Point MTR-4402: 6-hour unplanned
outage on 2026-07-10; SL...
```

grounded_reasoning_node is the LLM-backed upgrade to the Day 1 keyword heuristic. It reads **AZURE_AI_FOUNDRY_ENDPOINT** and **AZURE_AI_FOUNDRY_API_KEY** from environment variables (never hardcoded), and if both are set, sends the retrieved document text to your gpt-5 deployment with a strict system prompt instructing it to rely only on the retrieved context and respond with just an adjustment code. If the environment variables aren't set, it logs that it is skipping and falls back to whatever the Day 1 keyword heuristic already decided — the pipeline degrades gracefully rather than failing.

CODE

```
# =====
# Grounded Reasoning Node -- Azure AI Foundry
# Maps retrieved evidence to a standardized adjustment code (U01-U05) using
# a strictly-grounded system prompt. Requires AZURE_AI_FOUNDRY_ENDPOINT /
# AZURE_AI_FOUNDRY_API_KEY / MODEL_DEPLOYMENT_NAME to be set as environment
# variables in your own Azure AI Foundry project -- no key is embedded here.
# =====
import os
from openai import OpenAI

AZURE_ENDPOINT = os.getenv("AZURE_AI_FOUNDRY_ENDPOINT")
AZURE_API_KEY = os.getenv("AZURE_AI_FOUNDRY_API_KEY")
MODEL_DEPLOYMENT_NAME = os.getenv("MODEL_DEPLOYMENT_NAME", "gpt-5")

GROUNDED_SYSTEM_PROMPT = (
    "You are a utility billing adjustment-code classifier. Rely ONLY on the retrieved "
    "context provided. Map estimated/high meter readings to U01, outage credits to U02, "
    "estimated-bill corrections to U03, subsidy/discount adjustments to U04, and late-fee "
    "waivers to U05. Respond with only the code."
)

def grounded_reasoning_node(state: Dict[str, Any]) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))
    retrieved = state.get("retrieved_documents", [])
    context_text = "\n".join(d["text"] for d in retrieved)

    if not AZURE_ENDPOINT or not AZURE_API_KEY:
        audit_notes.append(
            "Grounded Reasoning skipped: AZURE_AI_FOUNDRY_ENDPOINT/API_KEY not configured in "
            "this "
            "environment. Falling back to the Notebook-1 keyword heuristic result."
        )
        updated_state["audit_notes"] = audit_notes
        return updated_state

    try:
        client = OpenAI(base_url=f"{AZURE_ENDPOINT.rstrip('/')}/openai/v1",
            api_key=AZURE_API_KEY)
        response = client.responses.create(
            model=MODEL_DEPLOYMENT_NAME,
            input=f"{GROUNDED_SYSTEM_PROMPT}\n\nContext:\n{context_text}",
        )
        llm_code = str(response.output[0]).strip()
        updated_state["adjustment_code"] = llm_code
        audit_notes.append(f"Grounded Reasoning (Azure AI Foundry) assigned code: {llm_code}.")
    except Exception as err:
        audit_notes.append(f"Grounded Reasoning call failed, keeping keyword-heuristic result: {err}")

    updated_state["audit_notes"] = audit_notes
    return updated_state
```

Cell 15 — Test the grounded reasoning node

Runs the node against the retrieved documents from Cell 13. Without Foundry credentials configured as environment variables you should see the graceful "skipped" message and the final adjustment code fall back to **U01** from the Day 1 heuristic; with valid credentials wired up, you would instead see the model's own grounded answer.

CODE

```
try:
    rag_state = grounded_reasoning_node(rag_state)
    print("Grounded Reasoning Result:")
    for note in rag_state["audit_notes"][-1:]:
        print(f" - {note}")
    print(f"Final Adjustment Code: {rag_state['adjustment_code']}")
except Exception as err:
    print(f"Grounded reasoning test failed: {err}")
```

OUTPUT

```
Grounded Reasoning Result:
 - Grounded Reasoning skipped: AZURE_AI_FOUNDRY_ENDPOINT/API_KEY not configured in this
environment. Falling back to the Notebook-1 keyword heuristic result.
Final Adjustment Code: U01
```

Cell 16 — Build the multi-node RAG StateGraph

Compiles a new, dedicated LangGraph graph for the RAG path: **process_remittance** → **vector_search** → **grounded_reasoning** → **deduce_adjustment** → **tolerance_write_off**. Putting Remittance Processing first means the keyword-based adjustment code is always available as a fallback, even if the grounded reasoning step is skipped for lack of credentials.

CODE

```
try:
    rag_builder = StateGraph(RAGUtilityState)
    rag_builder.add_node("process_remittance", remittance_processing_node)
    rag_builder.add_node("vector_search", vector_search_node)
    rag_builder.add_node("grounded_reasoning", grounded_reasoning_node)
    rag_builder.add_node("deduce_adjustment", adjustment_discrepancy_node)
    rag_builder.add_node("tolerance_write_off", tolerance_auto_write_off_node)

    rag_builder.add_edge(START, "process_remittance")
    rag_builder.add_edge("process_remittance", "vector_search")
    rag_builder.add_edge("vector_search", "grounded_reasoning")
    rag_builder.add_edge("grounded_reasoning", "deduce_adjustment")
    rag_builder.add_edge("deduce_adjustment", "tolerance_write_off")
    rag_builder.add_edge("tolerance_write_off", END)

    rag_app = rag_builder.compile()
    print("Multi-node RAG StateGraph compiled successfully.")
except Exception as err:
    print(f"Failed to compile RAG StateGraph: {err}")
```

OUTPUT

```
Multi-node RAG StateGraph compiled successfully.
```

Cell 17 — Execute the multi-node RAG pipeline end to end

Runs the compiled RAG graph against the Example B payload with the search query attached. You should see 2 retrieved documents, DOC-MER-001 as the top document, adjustment code U01, and a final state of PARTIAL MATCH — the same business outcome as Day 1's Example B, but now arrived at through vector retrieval and a grounded-reasoning node instead of a plain keyword scan.

CODE

```
try:
    rag_input_state = dict(create_initial_reconciliation_state(sample_payment_b,
sample_billing_b, sample_dispute_b))
    rag_input_state["search_query"] = "meter read estimated high dispute credit"
    rag_final_state = rag_app.invoke(rag_input_state)

    print("\n--- Multi-Node RAG Pipeline Execution Output ---")
    print(f"Retrieved Documents: {len(rag_final_state['retrieved_documents'])}")
    top_doc = rag_final_state["retrieved_documents"][0]["id"] if
rag_final_state["retrieved_documents"] else "N/A"
    print(f"Top Document: {top_doc}")
    print(f"Mapped Adjustment Code: {rag_final_state['adjustment_code']}")
    print(f"Final State: {rag_final_state['current_state']}")
except Exception as err:
    print(f"Multi-node RAG pipeline execution failed: {err}")
```

OUTPUT

```
--- Multi-Node RAG Pipeline Execution Output ---
Retrieved Documents: 2
Top Document: DOC-MER-001
Mapped Adjustment Code: U01
Final State: PARTIAL MATCH
```


`run_test_case` is a tiny assertion helper that prints PASS or FAIL for a named condition. This cell reruns five checks against state produced back in Notebook 1 — Example A closing on Priority Rule 1, Example B mapping to U01, Example C's tolerance write-off, Example D landing on UAC, and Smart Matching's confidence threshold — and prints a pass-rate summary. This is the pattern you would extend with your own test cases before trusting a change to the rule engine in production.

CODE

```
# =====
# B4: Testing & Evaluating Agents
# A lightweight assertion-based test harness (mirrors the capstone doc's
# Module 1-4 test cases) so agent behavior is regression-tested, not just
# eyeballed.
# =====
def run_test_case(name: str, condition: bool, detail: str = "") -> bool:
    status = "PASS" if condition else "FAIL"
    print(f"[{status}] {name}{'(' + detail if detail and not condition else ''}")
    return condition

test_results = []
test_results.append(run_test_case(
    "TC-1.1 Perfect 2-Way Match (Priority 1)",
    final_state_a["current_state"] == "CLOSED" and final_state_a["priority_rule_matched"] == 1,
))
test_results.append(run_test_case(
    "TC-1.2 Meter Dispute -> Code U01",
    state_b["adjustment_code"] == "U01" and state_b["current_state"] == "PARTIAL MATCH",
))
test_results.append(run_test_case(
    "TC-1.3 Tolerance Auto Write-Off (<= $5.00)",
    final_state_c["current_state"] == "CLOSED" and final_state_c["auto_write_off_amount"] ==
2.00,
))
test_results.append(run_test_case(
    "TC-4.1 Exception Routing -> UAC",
    state_d["current_state"] == "UAC",
))
test_results.append(run_test_case(
    "TC-Smart Suggests Probable Match Above Threshold",
    state_smart["smart_match_candidates"][0]["bill_id"] == "BILL-4471"
    and state_smart["smart_match_confidence"] >= 0.75,
))

pass_rate = sum(test_results) / len(test_results) * 100
print(f"\nTest Suite Summary: {sum(test_results)}/{len(test_results)} passed
({pass_rate:.0f}%).")
```

OUTPUT

```
[PASS] TC-1.1 Perfect 2-Way Match (Priority 1)
[PASS] TC-1.2 Meter Dispute -> Code U01
[PASS] TC-1.3 Tolerance Auto Write-Off (<= $5.00)
[PASS] TC-4.1 Exception Routing -> UAC
[PASS] TC-Smart Suggests Probable Match Above Threshold

Test Suite Summary: 5/5 passed (100%).
```

`deployment_manifest` is a plain dictionary describing how this graph would actually run in production — Azure Container Apps, autoscaling rules, which secrets it needs, and which message bus carries A2A hand-offs — deliberately kept as data rather than code, since a real deployment pipeline would read a file just like this one.

`operate_batch` then simulates the production "operate" loop by invoking the compiled Day 1 `app` across a small batch of transactions in sequence.

CODE

```
# =====
# B5: Deploying & Operating Agents
# A deployment manifest is data, not code -- describing how this graph
# would run in Azure Container Apps, independent of any one notebook cell.
# =====
deployment_manifest = {
    "service_name": "greengrid-reconciliation-agent",
    "compute": "Azure Container Apps",
    "min_replicas": 1,
    "max_replicas": 6,
    "scale_rule": "queue_depth > 50",
    "secrets": ["AZURE_AI_FOUNDRY_ENDPOINT", "AZURE_AI_FOUNDRY_API_KEY"],
    "secret_source": "Azure Key Vault (referenced, never inlined in code)",
    "observability": "Azure Monitor / Application Insights",
    "message_bus": "Azure Service Bus (A2A hand-off transport)",
}

print("Deployment Manifest (Azure Infrastructure Blueprint, A5):")
for k, v in deployment_manifest.items():
    print(f" {k}: {v}")

def operate_batch(transactions: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    # Simulates the 'operate' loop: processes a batch of transactions through
    # the compiled Day 1 graph, as a production scheduler would.
    results = []
    for state in transactions:
        try:
            final = app.invoke(state)
            results.append(final)
        except Exception as err:
            print(f"Batch item failed: {err}")
    return results

try:
    batch_results = operate_batch([initial_state_a, initial_state_c])
    print(f"\nBatch run processed {len(batch_results)} transactions. "
          f"End states: {[r['current_state'] for r in batch_results]}")
except Exception as err:
    print(f"Operate batch failed: {err}")
```

OUTPUT

```
Deployment Manifest (Azure Infrastructure Blueprint, A5):
  service_name: greengrid-reconciliation-agent
  compute: Azure Container Apps
  min_replicas: 1
  max_replicas: 6
  scale_rule: queue_depth > 50
  secrets: ['AZURE_AI_FOUNDRY_ENDPOINT', 'AZURE_AI_FOUNDRY_API_KEY']
  secret_source: Azure Key Vault (referenced, never inlined in code)
  observability: Azure Monitor / Application Insights
  message_bus: Azure Service Bus (A2A hand-off transport)
```

Batch run processed 2 transactions. End states: ['CLOSED', 'CLOSED']

`compute_observability_metrics` reads every JSON line already written to `system.log` by this and the previous notebook, counts how many transactions landed in each end state, and calculates the overall QUERY rate. This is the same telemetry stream you've been writing to all along — this cell is simply the first place in the capstone where you read it back as a dashboard instead of a raw log.

CODE

```
# =====
# B6: Observability Deep Dive
# Aggregate telemetry from system.log into simple operational metrics:
# end-state distribution and QUERY rate.
# =====

import json as _json
from collections import Counter

def compute_observability_metrics(log_path: str = "system.log") -> Dict[str, Any]:
    state_counter = Counter()
    total_events = 0

    with open(log_path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            try:
                event = _json.loads(line)
                total_events += 1
                to_state = event.get("state_transition", {}).get("to")
                if to_state:
                    state_counter[to_state] += 1
            except _json.JSONDecodeError:
                continue

    query_count = state_counter.get("QUERY", 0)
    query_rate = (query_count / total_events * 100) if total_events else 0.0

    return {
        "total_telemetry_events": total_events,
        "end_state_distribution": dict(state_counter),
        "query_rate_pct": round(query_rate, 2),
    }

try:
    metrics = compute_observability_metrics()
    print("Observability Dashboard Snapshot:")
    for k, v in metrics.items():
        print(f" {k}: {v}")
except Exception as err:
    print(f"Observability metrics computation failed: {err}")
```

OUTPUT

```
Observability Dashboard Snapshot:
total_telemetry_events: 6
end_state_distribution: {'CLOSED': 3, 'PARTIAL MATCH': 1, 'UAC': 1, 'INDEXED': 1}
query_rate_pct: 0.0
```

A final wrap-up cell listing everything this notebook added on top of Day 1: the Supervisor-orchestrated agents and Microsoft Agent Framework wrapper, the MCP tool server and A2A hand-off, the Voice Agent lab, the ChromaDB-backed retrieval and grounded reasoning node, the regression test suite, the deployment manifest and batch operate loop, and the observability metrics. It also previews what Notebook 3 adds next.

CODE

```
print("=" * 70)
print("DAY 2 CAPABILITIES BUILT")
print("=" * 70)
print("- B1 Supervisor-orchestrated agents + Microsoft Agent Framework wrapper")
print("- B2 MCP tool server (query_billing_record) + A2A message-contract handoff")
print("- B3 Voice Agent Lab: IVR transcript -> dispute note -> remittance pipeline")
print("- A4 Retrieval Augmented Agents: ChromaDB + grounded reasoning node")
print("- B4 Assertion-based regression test suite")
print("- B5 Deployment manifest + batch 'operate' loop")
print("- B6 Observability metrics computed from system.log")
print("\nNext: Notebook 3 adds security guardrails, PII masking, Partial Payments,")
print("D365 ERP journal-entry posting, and governance reporting (Day 3 / Track C).")
```

OUTPUT

```
=====
DAY 2 CAPABILITIES BUILT
=====
- B1 Supervisor-orchestrated agents + Microsoft Agent Framework wrapper
- B2 MCP tool server (query_billing_record) + A2A message-contract handoff
- B3 Voice Agent Lab: IVR transcript -> dispute note -> remittance pipeline
- A4 Retrieval Augmented Agents: ChromaDB + grounded reasoning node
- B4 Assertion-based regression test suite
- B5 Deployment manifest + batch 'operate' loop
- B6 Observability metrics computed from system.log

Next: Notebook 3 adds security guardrails, PII masking, Partial Payments,
D365 ERP journal-entry posting, and governance reporting (Day 3 / Track C).
```